Improving Rubik's Cube Solving with Neural-Guided Move Ordering in IDA*

Akash Chakraborty and Atul Chaudhuri

Abstract The Rubik's Cube has roughly 4.3×10^{19} reachable states, making brute-force solving impractical. We build on Kociemba's Two-Phase Algorithm—a well-known near-optimal solver—and extend it in three ways: (i) **multi-solution Phase 1 search**, which tries several intermediate paths and keeps the shortest overall solution; (ii) a **time-bounded anytime mode** for interactive use; and (iii) **neural move ordering** that guides the search toward promising moves using learned predictions. On 200 random cubes, multi-solution search cuts the average solution by 0.42 moves ($p < 0.001$) and doubles the share of $\leq$20-move solutions (28%$\rightarrow$56%). For neural ordering, we show that *how* the cube state is represented matters far more than model size: a simple change from 9 scalar features to 161 binned coordinate features raises prediction accuracy from 5.6% to 83.3% and halves the number of search nodes. However, the cost of running the neural network at every node wipes out the speed gain in Python. We resolve this with a precomputed lookup table (LUT) that replaces neural inference with a single array access, largely eliminating the overhead. All code, data, and trained models are provided for reproducibility.

Keywords: Rubik's Cube, two-phase algorithm, Kociemba, IDA*, pruning tables, neural heuristics, move ordering, anytime solver

Akash Chakraborty
Department of Computer Science and Engineering, Sister Nivedita University, e-mail: akashchakraborty694@gmail.com

Atul Chaudhuri
Department of Computer Science and Engineering, Sister Nivedita University, e-mail: atulc23@gmail.com

1 Introduction

1.1 The Rubik's Cube as a Search Problem

The Rubik's Cube, invented by Ernő Rubik in 1974, is one of the most widely studied combinatorial puzzles in computer science [22]. A standard $3\times3\times3$ cube has 26 visible sub-cubes ("cubies"): 8 corners, 12 edges, and 6 face centres. Each corner shows 3 coloured stickers and each edge shows 2, giving 54 stickers in total.

The six faces (Up, Down, Left, Right, Front, Back) can each be turned in three ways—clockwise, counter-clockwise, or a half-turn—yielding 18 possible moves at every step. The number of reachable configurations is

$$|G| = \frac{8! \cdot 3^8 \cdot 12! \cdot 2^{12}}{12} \approx 4.33 \times 10^{19}. \tag{1}$$

This enormous state space rules out any brute-force approach.

1.2 God's Number and Practical Solving

"God's Number" is the fewest moves needed to solve any configuration from the worst case. In the *half-turn metric* (HTM), where each quarter-turn or half-turn counts as one move, God's Number is **20**, proved by Rokicki et al. [2] in 2010. In other words, every scrambled cube *can* be solved in at most 20 moves—but finding those 20 moves requires searching an astronomically large space.

Kociemba's Two-Phase Algorithm [1] offers a practical middle ground. It does not guarantee exactly 20 moves for every cube, but it reliably produces 19–22 move solutions in under a second. This paper implements Kociemba's algorithm and contributes three extensions:

1. **Multi-solution Phase 1 search** ($K > 1$): instead of accepting the first intermediate result, the solver explores K alternatives and keeps the shortest combined solution.
2. **Time-bounded anytime mode**: the solver returns the best solution found so far when a time limit is reached, enabling responsive interactive applications.
3. **Neural move ordering**: a small neural network predicts which moves are most promising, and the search tries those first. We show that input representation is the key factor, not model size.

We also provide formal analysis of why naïve neural integration fails (Theorem 1), why multi-solution search helps (Proposition 1), and what determines whether neural ordering speeds up or slows down the solver (Theorem 2).

1.3 Paper Organisation

Section 2 reviews prior work. Section 3 gives the mathematical background. Section 4 describes our methodology. Sections 5–6 present and discuss results. Section 7 concludes.

2 Related Work

Optimal solvers. Korf [5] built the first optimal Rubik's Cube solver using IDA* [12] with pattern databases [15], later extended to disjoint [16] and additive [10] variants. Rokicki et al. [2] proved God's Number is 20 using 35 CPU-years of computation. Demaine et al. [28] established worst-case bounds for generalised Rubik's Cubes.

Two-phase methods. Thistlethwaite [6] introduced nested-subgroup reduction with four phases. Kociemba [1] simplified this to two phases with coordinate-based pruning tables, dramatically improving speed. Human speedsolving methods like CFOP [11] use a similar multi-phase strategy but rely on pattern recognition rather than exhaustive search. Our work extends Kociemba's algorithm with multi-solution search and neural ordering.

Deep learning for the cube. DeepCubeA [4] uses deep reinforcement learning with weighted A* to find optimal solutions in 60% of cases, but requires $\sim$2 days of GPU training and $\sim$10 s per solve. Brunetto and Trunda [7] showed that feature representation is critical for distance estimation, consistent with our findings. More broadly, neural-guided tree search has been transformative in games like Go [17, 18].

Neural heuristics in planning. Shen et al. [8] showed that small heuristic errors cause exponential node growth in IDA*—the "heuristic noise" problem that our Theorem 1 formalises. Arfaee et al. [19] proposed bootstrapped learning of heuristic functions for large state spaces. Ferber et al. [9] demonstrated the importance of domain-specific features for neural heuristics, supporting our finding that binned coordinate features dramatically outperform raw scalars.

Positioning. Unlike end-to-end learned solvers, we study neural heuristics as *add-ons* to an established classical algorithm, inspired by move-ordering techniques in game-tree search [20, 21].

3 Mathematical Foundation

3.1 Coordinate System

Rather than storing full permutation arrays, the solver encodes each cube state using a small number of *coordinates*—integers that capture the essential structure:

- **Twist** (0–2186): how the 8 corners are oriented.
- **Flip** (0–2047): how the 12 edges are oriented.
- **Slice** (0–494): which 4 of the 12 edge positions hold middle-layer edges.
- **URFtoDLF** and **URtoDF**: corner and edge permutation coordinates used in Phase 2.

The key advantage of coordinates is speed: applying a move just requires looking up $c' = \text{moveTable}[c][m]$, an $O(1)$ table lookup.

3.2 How the Two-Phase Algorithm Works

The algorithm solves the cube in two stages (Fig. 1):

Phase 1 searches for a sequence of moves that brings the cube from its scrambled state into a special subgroup

$$G_1 = \langle U, D, L^2, R^2, F^2, B^2 \rangle. \tag{2}$$

A cube is in G_1 when all edges are correctly oriented (flip $= 0$), all corners are correctly oriented (twist $= 0$), and the four middle-layer edges are in middle-layer positions (slice $= 0$). Phase 1 uses all 18 moves.

Phase 2 takes the cube from the G_1 state to the solved state (the identity). Only 10 moves are allowed (the generators of G_1), so the effective branching factor drops from 18 to 10 and stronger pruning tables apply. Phase 2 is typically much faster than Phase 1.

Both phases use **IDA*** (Iterative Deepening A*) [12], a memory-efficient extension of A* [13]. IDA* performs repeated depth-first searches with an increasing cost limit. At each node, a heuristic lower bound h [14] (read from precomputed *pruning tables*) estimates the remaining moves. If the current depth g plus h exceeds the limit, the branch is pruned. The critical property for our analysis: the number of nodes grows *exponentially* with the depth limit, so adding even one extra level of search multiplies the work by $\sim$13.

3.3 Pruning Tables

Pruning tables store the minimum moves needed to reach the target (the G_1 subgroup in Phase 1, or the identity in Phase 2) from each coordinate value. They are built once by backward breadth-first search and cached on disk ($\sim$20 MB). The heuristic at each node is the maximum of two independent lower bounds (e.g., Slice–Twist and Slice–Flip for Phase 1), which tightens the estimate while remaining admissible.

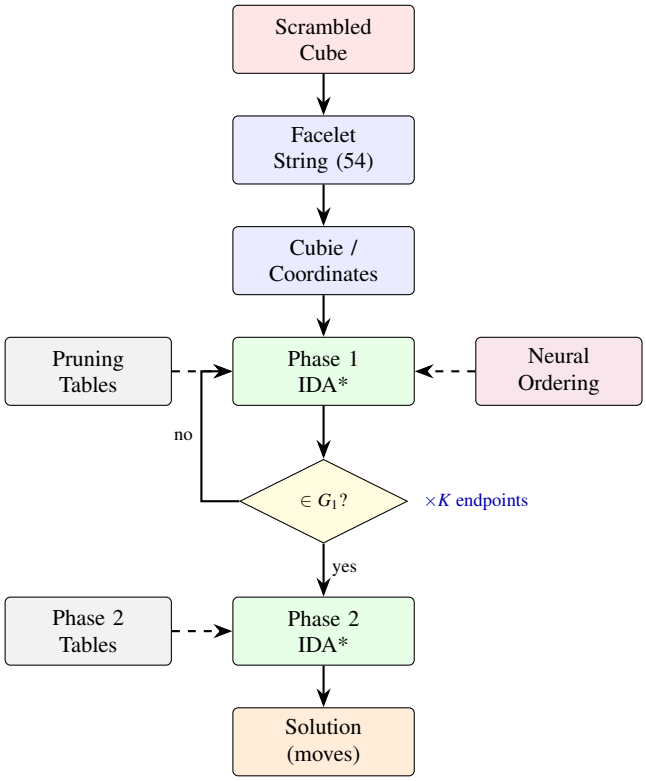

Fig. 1: Solver pipeline. The scrambled cube is converted to coordinates, then Phase 1 searches for states in subgroup G_1 (optionally guided by neural ordering). Phase 2 completes the solve. With $K > 1$, multiple Phase 1 endpoints are tried and the shortest overall solution is kept.

3.4 Consecutive Move Pruning

At each node the solver considers all 18 moves, but two simple rules eliminate redundant branches without losing solutions: (1) never apply two moves of the *same* face in a row (e.g., U then U' is just $U2$ or the identity), and (2) for opposite faces that commute (U/D, L/R, F/B), impose a fixed ordering to avoid duplicates ($UD = DU$). These rules reduce the average branching factor from 18 to about 13.35—a $\sim$500$\times$ reduction in tree size at depth 20.

3.5 Formal Results

We now state three results that guide our design choices.

Theorem 1 (Admissible Bound Paradox). *Let h^* be the optimal admissible heuristic for IDA*, and let $\hat{h} = h^* + \varepsilon$ be a neural heuristic that sometimes overestimates by $\varepsilon > 0$. Then IDA* with $\hat{h}$ expands at least $b_{\text{eff}}^{\lceil \varepsilon \rceil}$ times more nodes than IDA* with h^*.*

Why this matters. A natural idea is to train a neural network to predict how far the cube is from solved and use that as a tighter lower bound. Theorem 1 says this backfires: even a small overestimate forces IDA* to run extra iterations, each exponentially more expensive than the last. With our branching factor of $\sim$13.35, an overestimate of just 1 move causes a $\geq$13$\times$ slowdown.

Proof. IDA* expands $O(b_{\text{eff}}^d)$ nodes at threshold d. An admissible heuristic sets the initial threshold at $d_0 \leq d^*$ (the optimal depth). An overestimating heuristic prunes parts of the optimal path, forcing extra iterations up to threshold $d^* + \lceil \varepsilon \rceil$. The final iteration costs $O(b_{\text{eff}}^{d^* + \lceil \varepsilon \rceil})$, a factor $b_{\text{eff}}^{\lceil \varepsilon \rceil}$ more. □

Implication. The correct way to use neural predictions in IDA* is for *move ordering*—deciding which child to explore first—not for tightening the lower bound.

Proposition 1 (Multi-Solution Order-Statistic Bound). *If K Phase 1 endpoints lead to independent solution lengths with mean μ and standard deviation σ, the expected length of the best solution is approximately $\mu - \sigma \cdot c_K$ [25], where $c_K > 0$ grows slowly with K (for K=5, $c_K \approx 1.16$).*

In plain terms: trying multiple Phase 1 paths and keeping the best one gives shorter solutions, with diminishing returns as K grows.

Theorem 2 (Node Reduction from Neural Move Ordering). *If a neural predictor places the best move first with probability p at each level of the search tree, the expected number of nodes shrinks by a factor α^d where $\alpha = 1 - p(1 - 1/b)$ and d is the search depth. For random ordering ($p = 1/b$), $\alpha = 1$ (no benefit). For perfect prediction ($p = 1$), $\alpha = 1/b$ (only the optimal branch is explored).*

With our coordinate features achieving $p = 0.833$ and $b \approx 13$, the predicted per-level factor is $\alpha \approx 0.23$, qualitatively consistent with our empirically observed 2.4$\times$ reduction.

4 Methodology and Implementation

4.1 System Overview

The solver, built on the `kociemba` Python package [3], uses two representations: a **facelet string** (54 characters, one per sticker) for input/output, and **coordinates** for search. Translation happens once per solve. Moves are applied via precomputed table lookups, making each move $O(1)$.

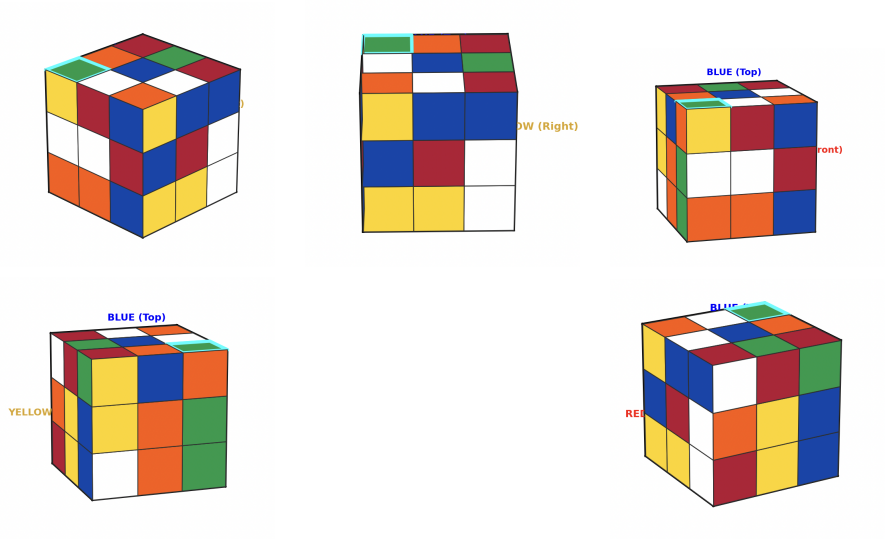

Fig. 2: Example snapshots from face selection and guided sticker entry in the 3D interface.

4.2 User Flow: 3D Cube State Input

The application provides an interactive 3D cube interface for entering the physical cube's state. The end-to-end user flow is as follows:

1. **Launch.** The user starts the application and is presented with a 3D rendering of a solved Rubik's Cube (Matplotlib-based, using `Poly3DCollection`). A 2D unfolded net, a colour palette, and an information panel are displayed alongside the 3D view.
2. **Orient the physical cube.** The user holds the physical cube in a fixed orientation: *white center on top, blue center facing front*. This matches the standard colour scheme assumed by the solver (U = white, F = blue, R = red, D = yellow, L = orange, B = green).
3. **Select a face.** The user navigates to a face either by clicking on the 2D net, or by pressing `Tab` / `Shift+Tab` to cycle through the six faces in the order $U \to F \to R \to B \to L \to D$. A *guided mode* (toggled with `G`) walks the user through each face and sticker sequentially, which is helpful for first-time users (Fig. 2).
4. **Paint stickers.** For each face, the user sets each of the 8 non-center stickers to the colour visible on the physical cube. Input methods:
   - *Click:* click a sticker on the 2D net, then click the desired colour in the palette.

- *Keyboard:* use arrow keys to move the selection highlight, then press `1–6` (white, red, blue, yellow, orange, green).

Center stickers are locked to their face's identity colour and cannot be changed, preventing a common input error.

5. **Visual feedback.** Both the 3D cube and the 2D net update in real time as stickers are painted. The user can drag-rotate the 3D view to visually verify hidden faces. A status bar shows the count of each colour; valid input requires exactly 9 stickers per colour.
6. **Solve.** Once all 54 stickers are set, the user presses `Enter`. The interface converts the face grid into a 54-character facelet string (e.g., `UUUUURRRR...`), validates it against the Kociemba solver's facelet constraints, and invokes the two-phase solver. The solution sequence is displayed on screen.
7. **Reset / Correct.** At any point the user can press `R` to reset the cube to the solved state and start over, or simply repaint individual stickers to correct mistakes.

This flow replaces the earlier camera-based colour detection pipeline, eliminating lighting- and calibration-dependent errors and giving the user full control over the input state.

4.3 Multi-Solution Phase 1 Search

The baseline Kociemba solver returns the *first* Phase 1 solution and runs Phase 2 once. But different Phase 1 paths reach different points in the G_1 subgroup, and the Phase 2 distance from each can vary significantly. Our extension introduces a parameter K: the solver finds up to K Phase 1 endpoints, runs Phase 2 from each, and keeps the shortest overall solution. For $K{=}1$ the behaviour is identical to the baseline. For $K > 1$, the solver trades compute time for shorter solutions (Algorithm 1, Fig. 3).

4.4 Time-Bounded Anytime Mode

For interactive applications, unbounded solve time is unacceptable. We add an optional time budget T (e.g., 0.5 s). Before each Phase 1 iteration the solver checks the clock; if time is up and at least one solution exists, it returns the best solution found so far. The quality is monotonically non-decreasing: once a solution is found (typically within 100 ms), it can only be improved or kept, never worsened.

Algorithm 1 Multi-Solution Phase 1 IDA*

Require: Cube state s_0, max solutions K, time budget T
Ensure: Best solution found

```
d_limit ← h_1(s_0)
best ← None; best_len ← ∞; count ← 0
while count < K and elapsed < T do
    t ← DFS-PHASE1(s_0, 0, d_limit)
    if t = FOUND then
        continue
    else if t = ∞ then
        break {No more solutions}
    else
        d_limit ← t {Raise threshold}
    end if
end while
return best
```

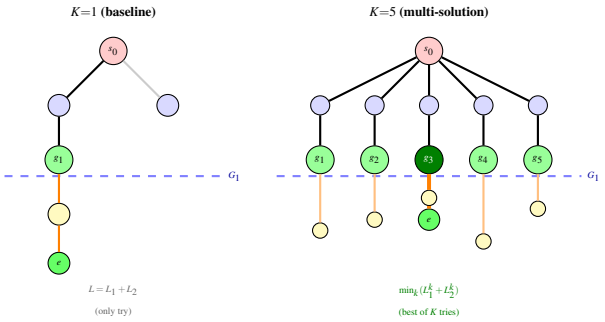

Fig. 3: Multi-solution Phase 1. Left: K=1 finds one G_1 endpoint and accepts whatever Phase 2 gives. Right: K=5 finds five endpoints, runs Phase 2 from each, and returns the shortest total (dark green, g_3). Mean improvement: -0.42 moves ($p < 0.001$).

4.5 Neural Move Ordering

The core idea. At each search node, instead of trying the 18 moves in a fixed order, we ask a neural network: "which moves look most promising?" and try those first. Because IDA* explores *all* children at each non-pruned node regardless of order, this does not change which solutions are found—only how quickly we stumble onto the good path. The search remains complete and optimal within each depth iteration.

Why naïve neural bounds fail. A tempting alternative is to use the network's prediction as a tighter lower bound to prune more aggressively. Theorem 1 explains why this fails: any overestimate forces extra iterations, each exponentially expensive. We observed up to 133× slowdown in practice.

Three strategies. We evaluate three ways to integrate neural ordering, all leaving the pruning bound unchanged:

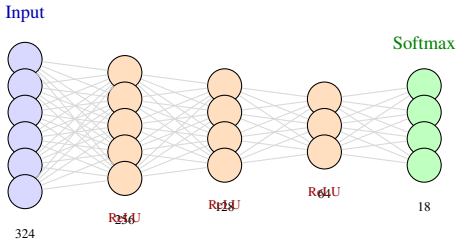

Fig. 4: Move predictor MLP. The one-hot encoding (324 dims) or coordinate features (161 dims) pass through three hidden layers. The 18-way softmax output gives a distribution over moves for ordering.

A. Always order: query the network at every node.
B. Probabilistic: use neural ordering with probability p, default order otherwise.
C. Depth cutoff: use neural ordering only at shallow depths ($\leq D_{\text{cut}}$).

4.6 Neural Network Architecture

We train two models:

Move predictor (π_θ): an MLP (Fig. 4) mapping the cube state to a distribution over 18 moves ($324 \to 256 \to 128 \to 64 \to 18$, ReLU activations, softmax output). The input is a 54-sticker one-hot encoding (54 positions $\times$ 6 colours = 324 dimensions) or a 161-dimensional coordinate feature vector (described in Section 5.6).

Value predictor (v_ϕ): an MLP estimating moves-to-solve ($324 \to 512 \to 256 \to 128 \to 1$). Used only to analyse the admissible bound paradox; *not* used in any strategy.

Training. We generate 50,000 training cubes by applying k random moves ($k \sim \text{Uniform}[1, 25]$) to the solved state. The label is the reverse of the last scramble move, a simple proxy for the "best" move. Training: batch size 64, Adam [27] ($\text{lr} = 10^{-3}$), 30 epochs ($\sim$5 min on CPU). We also tested solver-feedback labels (the Kociemba solver's actual first move), which are more expensive to generate ($\sim$2 hours for 50K samples).

4.7 Experimental Setup

Hardware. All experiments ran on a consumer laptop (Apple M-series, 16 GB RAM) using Python 3.11, NumPy 1.26, PyTorch 2.1 [29].

Table 1: C backend vs. Python backend (200 scrambles, seed 42, $K=1$).

Backend	Mean moves	$\leq$20 (%)	Mean ms	P95 ms	Max ms	Speedup
C	20.77	28.0	11.8	36.1	66.0	59.4×
Python	20.77	28.0	698.5	2045.9	3537.2	1×

Scrambles. 200 per experiment, generated by applying 25 random moves with explicit random seeds (primary: 42; validation: 123, 456, 789). Hard cases: 50 cubes including the superflip and 49 depth-20 scrambles.

Configuration. 5 s timeout per solve; pruning tables precomputed and cached ($\sim$30 s, $\sim$20 MB). All strategy comparisons use the same scrambles (same seed), enabling paired statistical tests.

Metrics. For each solve: total nodes expanded, Phase 1/Phase 2 nodes, max depth, solution length, and wall-clock time. We report paired t-tests, Cohen's d effect sizes [23], and 95% bootstrap confidence intervals [24].

4.8 Limitations

Several limitations should be noted: (1) all experiments use the pure Python solver for reproducibility—the C backend is 59× faster but produces identical solutions; (2) our MLP has only $\sim$50K parameters, three orders of magnitude smaller than DeepCubeA; (3) training data (50K–100K cubes) is tiny relative to the 4.3×10^{19} state space; (4) the solver is single-threaded.

5 Results

5.1 Baseline Performance

Table 1 confirms algorithmic equivalence between the two backends: identical solutions on all 200 scrambles, with the C backend 59× faster.

The baseline ($K=1$) produces solutions of 18–22 moves, with a mean of 20.77 and 28% at $\leq$20 moves. The distribution is dominated by 21-move solutions (117/200 = 58.5%).

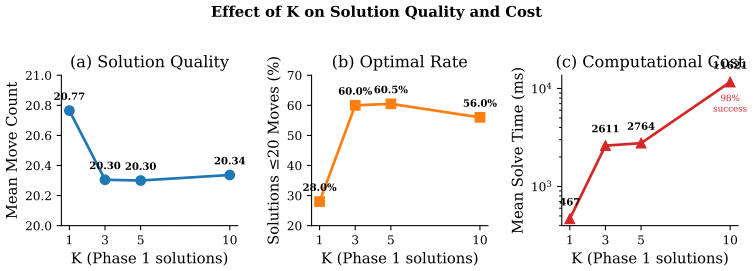

Fig. 5: Effect of K on solution quality, optimality rate ($\leq$20 moves), and solve time. Quality improves sharply from K=1 to K=3, then plateaus.

Table 2: Move count and solve time vs. K (200 scrambles, seed 42, 5 s timeout).

Setting	**Mean**	**Median**	**SD**	**Min–Max**	**≤20 (%)**	**Mean time (ms)**	**Max time (ms)**
Baseline ($K=1$)	20.77	21	0.81	18–22	28.0	648	3410
Multi ($K=3$)	20.36	20	0.76	18–22	54.5	2814	5000
Multi ($K=5$)	20.35	20	0.76	18–22	56.0	2843	5000
Multi ($K=10$)	20.34	20	0.75	18–22	56.0	2835	5000

5.2 Multi-Solution Phase 1 Results

Table 2 and Fig. 5 show that exploring multiple Phase 1 endpoints significantly improves solution quality.

Key findings:

- K=5 reduces the mean by 0.42 moves (20.77→20.35) and doubles $\leq$20-move solutions from 28% to 56% (Fig. 6).
- K=3 already captures most of the benefit (20.36 mean, 54.5% $\leq$20). Going to K=10 adds nothing (K=5 and K=10 are essentially identical).
- Mean solve time rises from 648 ms to 2843 ms (4.4×), all within the 5 s timeout.
- Paired test (K=1 vs. K=5): $t = 9.86$, $p < 0.001$, Cohen's $d = 0.70$ (medium-to-large), 95% CI $[0.33, 0.50]$ moves.

The move distribution (Table 3) reveals how multi-solution search works: it primarily converts 21-move solutions into 20-move solutions, while the 22-move tail shrinks from 27 to 6.

5.3 Anytime Mode Results

Table 4 and Fig. 7 show the quality-latency trade-off of anytime mode (K=5, varying time budget).

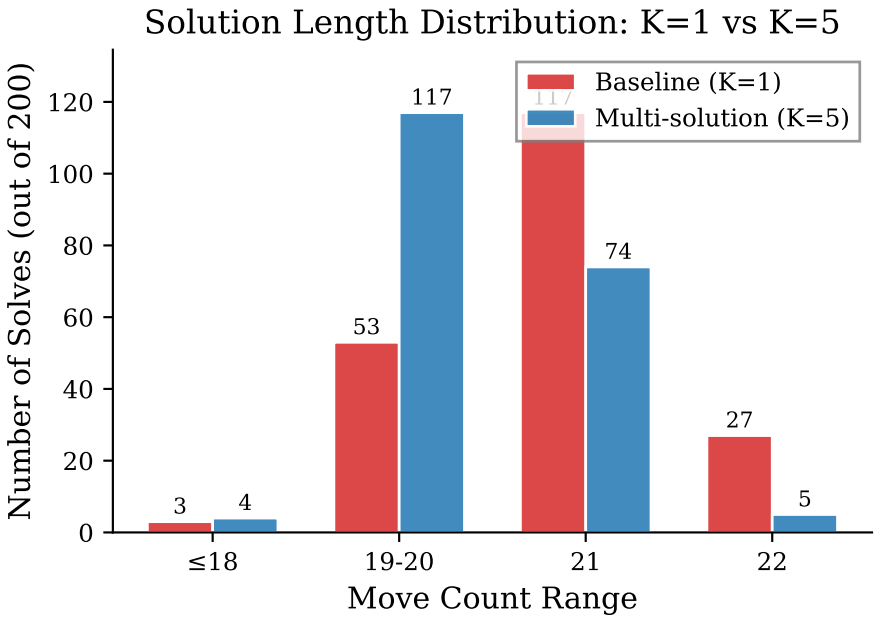

Fig. 6: Solution length distribution: K=1 vs. K=5. Multi-solution search shifts mass from 21–22 moves toward ≤20 moves.

Table 3: Move-length distribution across K (200 scrambles, seed 42).

K	≤18	19–20	21	22	≤20 (%)
1 (BL)	3	53	117	27	28.0
3	4	105	85	6	54.5
5	4	108	82	6	56.0
10	4	108	83	5	56.0

An important subtlety: the 0.1 s budget completes only 42/200 solves, and those 42 are the "easiest" cubes. Their mean of 20.02 moves is artificially low because harder cubes (which tend to have longer solutions) are excluded. As the budget

Table 4: Anytime mode: solution quality vs. time budget (200 scrambles, seed 42, K=5).

Budget (s)	Mean moves	≤20 (%)	Mean (ms)	P95 (ms)	P99 (ms)	Success
0.1	20.02	69.0	59.3	100.2	100.5	42/200
0.3	20.37	48.9	205.1	300.1	300.2	94/200
0.5	20.42	45.9	353.8	500.1	500.1	133/200
1.0	20.43	49.1	673.9	1000.1	1000.1	173/200

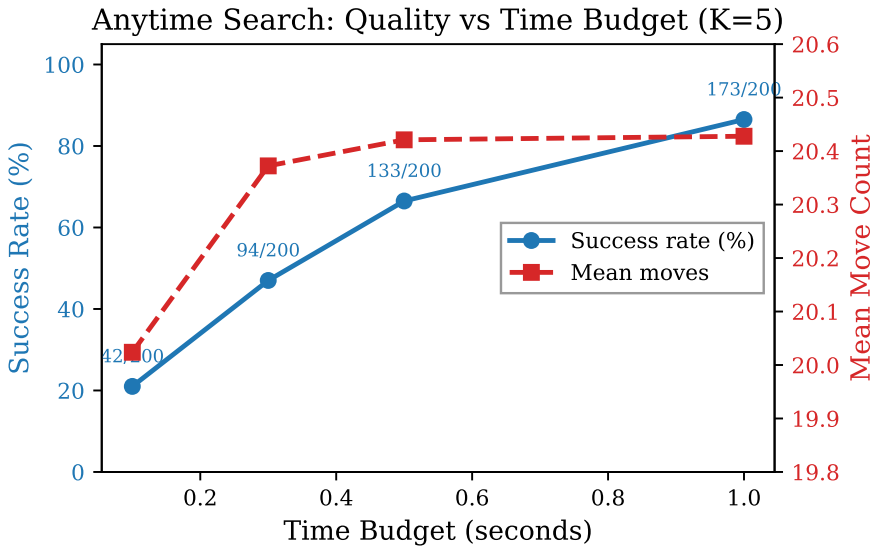

Fig. 7: Anytime mode: success rate and mean moves vs. time budget.

grows, harder cubes enter the pool and the mean rises slightly. A 0.5 s budget gives 66.5% completion with P95 at 500 ms—a reasonable choice for interactive use.

5.4 Node Expansion Analysis

Table 5 shows that multi-solution search expands 5.4× more nodes, with the increase concentrated in Phase 1 (6.0×). Phase 2 increases only 1.5× because each Phase 2 search is inherently cheap. The neural strategies with scalar features produce virtually identical node counts (within 0.5%), confirming that the scalar-feature predictor outputs near-uniform distributions that do not reorder the search.

Table 5: Mean nodes per solve (200 scrambles, seed 42).

Setting	Total	Phase 1	Phase 2
$K = 1$ (baseline)	787 601	691 168	96 433
$K = 5$ (no neural)	4 280 549	4 139 705	140 845
$K = 5$ + neural A	4 260 514	4 119 767	140 747
$K = 5$ + neural B	4 259 539	4 118 890	140 649
$K = 5$ + neural C	4 290 369	4 149 434	140 935

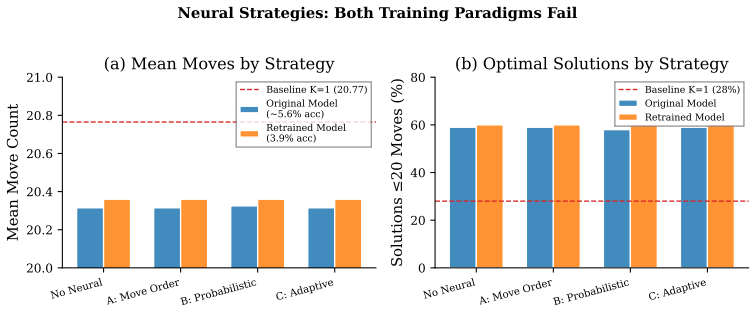

Fig. 8: Neural strategy comparison. Neither scalar-feature model nor solver-feedback retrained model improves over the non-neural baseline. Error bars: 95% bootstrap CIs.

5.5 Neural Strategy Results: Scalar Features Fail

With scalar features (9 dimensional normalised coordinates), all three neural strategies produce *identical* solution quality to the non-neural K=5 baseline (Table 6, Fig. 8). The predictor's accuracy is 5.6%—statistically indistinguishable from random guessing ($1/18 \approx 5.56\%$). The model has learned nothing useful.

The strategies only add overhead: +18% for A, +51% for B, +0% for C. Strategy C has near-zero cost because it only queries the network at shallow depths, which are a tiny fraction of total nodes.

We also retrained with solver-feedback labels (the actual first move the solver chooses), expecting better signal. Surprisingly, accuracy *dropped* to 3.9%—below random. We attribute this to label inconsistency: the solver's first move depends on

Table 6: Neural strategies with scalar features (200 scrambles, K=5).

Strategy	Mean	≤20	ms	P95
K=1 baseline	20.77	28.0%	500	1442
K=5 none	20.35	56.0%	2650	5000
K=5 + A	20.35	56.0%	3126	5000
K=5 + B (p=.5)	20.35	55.5%	3997	5000
K=5 + C (D=5)	20.35	56.0%	2649	5000

internal search dynamics, producing a multi-modal label distribution that a simple softmax struggles with.

Diagnosis. The problem is not the integration strategy (all three preserve IDA* completeness) nor the training signal. The problem is the **representation**: 9 normalised scalar coordinates destroy the discrete structure that the solver's pruning tables exploit.

5.6 Coordinate Features: Breaking the Representation Barrier

Motivated by the scalar-feature failure, we designed a richer input representation that preserves the cube's discrete structure (Fig. 9):

- **Binned one-hot** (144 dims): each coordinate is discretised into bins and one-hot encoded (32, 32, 16, 32, 32 bins).
- **Parity** (2 dims): even/odd permutation parity.
- **Scalar** (9 dims): original normalised coordinates.
- **Interactions** (6 dims): pairwise products of key coordinates.

Table 7 shows the dramatic impact.
Key findings:

1. **Accuracy jumps from 5.6% to 83.3%**—a 15× improvement, confirming the representation bottleneck.
2. **Nodes drop by 2.4**× (2.94M→1.21M), validating Theorem 2.

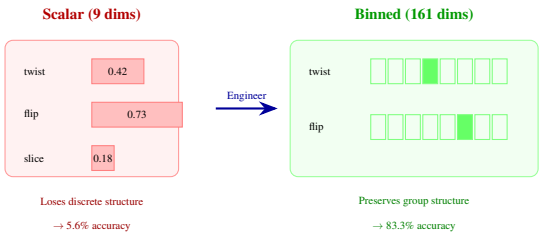

Fig. 9: Feature representation comparison. Scalar coordinates normalise to $[0,1]$, destroying discrete structure (5.6% accuracy). Binned one-hot encoding preserves structural regions (83.3% accuracy)—a 15× improvement with the same MLP.

Table 7: Coordinate features: accuracy and search impact (200 scrambles, K=5).

Strategy	**Dims**	**Acc.**	**Solved**	**Mean moves**	**Nodes**	**Time (ms)**
K=5 no neural	—	—	200	20.35	2.94M	2932
K=5 + A (move_order)	161	83.3%	173	20.45	1.21M	3276
K=5 + B (probabilistic)	161	—	189	20.49	1.67M	3125
K=5 + C (adaptive)	161	—	200	20.45	2.65M	3130

3. **But wall-clock time *increases* by 12%** (2932→3276 ms). The per-node cost of Python inference exceeds the time saved from fewer nodes. Even worse, 27/200 scrambles time out with Strategy A vs. 0/200 without neural ordering.
4. **Strategy C is safest**: comparable quality, no timeouts, negligible overhead.

The inference overhead barrier. Let c_{node} be the cost of a regular node expansion and c_{neural} be the cost of a neural forward pass. Neural ordering helps only when

$$N_{\text{neural}} \cdot (c_{\text{node}} + c_{\text{neural}}) < N_{\text{default}} \cdot c_{\text{node}}. \tag{3}$$

With our 2.4× node reduction, this requires $c_{\text{neural}} < 1.4 \cdot c_{\text{node}}$. In Python, $c_{\text{neural}} \approx 3 \cdot c_{\text{node}}$—too expensive.

5.7 Architecture Experiments

Is the barrier simply a matter of using a bigger model? Table 8 says no.

Tripling the parameters gains only +3.5 percentage points accuracy but adds +26% inference cost. The barrier *worsens* with larger models—this is not a capacity problem.

Table 8: Larger models: more accuracy at higher cost (100K samples, coord. features).

Model	**Params**	**Val Acc.**	**Top-3**	**Infer. (μs)**
MLP-3 (256→128→64)	84K	10.4%	25.8%	15.6
MLP-4 Wide (512→256→128→64)	257K	13.9%	28.2%	19.7
MLP-5 Deep (256^3→128→64)	215K	12.8%	28.1%	21.3

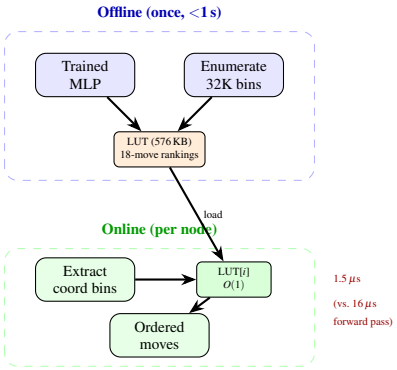

Fig. 10: LUT pipeline. **Offline**: enumerate all 32K coordinate bins, run the MLP once for each, store 18-move rankings (576 KB). **Online**: each node does an $O(1)$ array lookup instead of a neural forward pass.

5.8 LUT Precomputation: Resolving the Barrier

Instead of making inference faster, we *eliminate* it. We precompute move orderings for all $32 \times 32 \times 16 \times 2 = 32{,}768$ Phase 1 coordinate-bin combinations into a 576 KB lookup table—effectively a form of knowledge distillation [26]—built in $<$1 s (Fig. 10). During search, each node does an $O(1)$ array lookup ($\sim$1.5 μs) instead of a neural forward pass ($\sim$16 μs).

Results (Table 9, Fig. 11): K=5 + LUT (3047 ms) is 1.10$\times$ faster than per-node inference (3374 ms) and within 7% of the non-neural baseline (2843 ms), with 98.5% reliability vs. 94.0% for move_order. The LUT's node reduction (1.31$\times$) is smaller than full per-node inference (2.34$\times$) because the binned lookup loses fine-grained coordinate information, but the drastically reduced overhead makes the trade-off worthwhile.

Table 9: LUT vs. per-node neural inference (200 scrambles, seed 42, K=5).

Strategy	Solved	Mean	≤20 (%)	Nodes	Time (ms)
Baseline K=1	200	20.77	28.0	788K	648
K=5 none	200	20.35	56.0	3.39M	2843
K=5 + move_order	188	20.54	46.8	1.45M	3374
K=5 + LUT	197	20.45	49.7	2.59M	3047

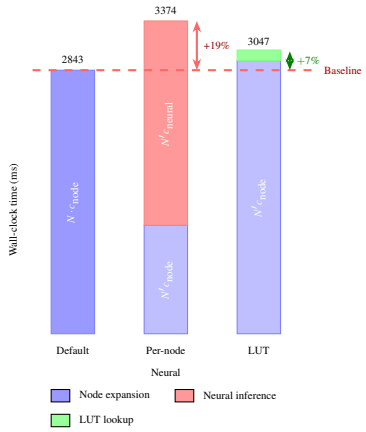

Fig. 11: Inference overhead barrier. Default (2843 ms): node expansion only. Per-node neural (3374 ms): 2.4× fewer nodes but +19% overhead from inference cost. LUT (3047 ms): same idea, but $O(1)$ lookup replaces the forward pass, cutting overhead to +7%.

5.9 Hard Cases and Multi-Seed Validation

Hard cases. We tested 49 depth-20 scrambles (the superflip timed out under both settings). K=5 improves ≤20-move solutions from 45% to 71% at a cost of 71× longer solve time (Table 10, Fig. 12).

Multi-seed validation. We repeated K=1 vs. K=5 across four seeds (200 scrambles each). The improvement is consistent: $\Delta = 0.40 \pm 0.09$ moves, with K=5 ≤20 rates of 51.5%–60.5% (Table 11, Fig. 13). Pooling all 800 pairs: $p < 0.001$, Cohen's $d = 0.59$, 95% CI $[0.35, 0.45]$.

Table 10: Hard cases: K=1 vs. K=5 (49 solves, seed 42).

Setting	Mean	Median	≤20	Mean ms
K=1 (baseline)	20.51	21	22/49 (45%)	435
K=5 (multi)	20.06	20	35/49 (71%)	31133

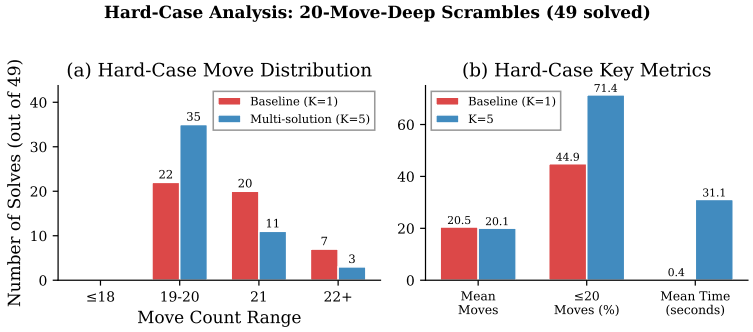

Fig. 12: Hard cases (K=1 vs. K=5): move distribution and key metrics.

Table 11: Multi-seed validation (200 scrambles each).

Seed	BL Mean	K5 Mean	Δ	K5 ≤20 (%)
42	20.77	20.30	0.47	60.5
123	20.85	20.36	0.49	57.5
456	20.67	20.31	0.36	57.0
789	20.74	20.45	0.29	51.5
Mean ± SD	20.75 ± 0.07	20.35 ± 0.07	0.40 ± 0.09	56.6 ± 3.8

6 Discussion

6.1 Why Multi-Solution Search Works

Different Phase 1 solutions reach different points in G_1, and the Phase 2 distance back to the identity varies. The first Phase 1 solution is essentially random among

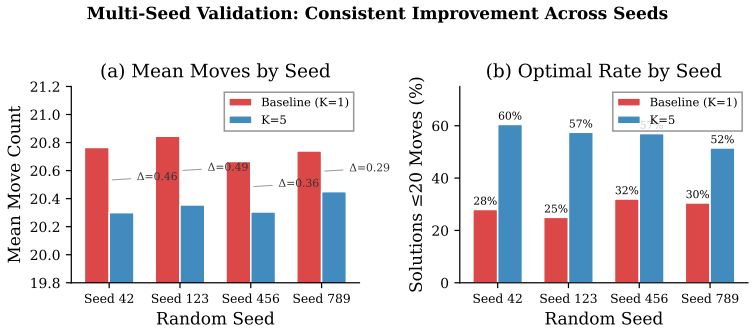

Fig. 13: Multi-seed validation. K=5 consistently outperforms the baseline across all four seeds, roughly doubling the $\leq$20-move rate.

those at the minimum depth; trying K of them and keeping the best is like taking the minimum of K draws from the Phase 2 length distribution.

The diminishing returns beyond K=3 indicate that Phase 2 lengths have limited variance: most endpoints lead to similarly long Phase 2 solutions, so additional samples have decreasing marginal value. Under the 5 s timeout, K=10 and K=5 produce identical results—the solver exhausts available Phase 1 endpoints regardless.

6.2 Three Levels of Neural Failure—and How We Fixed Two

Our investigation reveals a layered explanation:

Level 1: Low accuracy. With scalar features (9 dims), the predictor achieves 5.6%—random among 18 moves. This is the immediate cause of failure.

Level 2: Representation bottleneck (resolved). Switching to 161-dimensional binned features raises accuracy to 83.3%. The bottleneck was the input, not the model: the same MLP architecture suffices when given structurally informative inputs. Binned encoding preserves the discrete group-theoretic structure that scalar normalisation destroys.

Level 3: Inference overhead barrier (largely resolved). Even with 83.3% accuracy and 2.4$\times$ node reduction, per-node inference in Python is too expensive (Eq. 3). Architecture scaling makes this *worse* (Table 8). LUT precomputation largely *eliminates* it: per-node cost drops from $\sim$16 μs to $\sim$1.5 μs, and overhead drops from $+$19% to $+$7% of the non-neural baseline.

Bottom line. The overhead barrier is an engineering constraint, not a fundamental limit. Compiled C/CUDA inference or finer-grained LUTs would further close the remaining gap.

Table 12: Recommended settings for different use cases.

Scenario	Setting	Mean ms	Mean moves
C backend (fastest)	K=1, no budget	11.8	20.77
Real-time (Python)	K=1, no budget	648	20.77
Balanced	K=3, no budget	2814	20.36
Best quality	K=5, no budget	2843	20.35
Neural + LUT	K=5 + LUT	3047	20.45

6.3 Comparison with DeepCubeA

DeepCubeA [4] uses a much larger ResNet ($\sim$500K parameters), trains for 2 days on GPUs with millions of samples, and uses weighted A* that expands far fewer nodes ($\sim$1K vs. our $\sim$1M). This makes per-node neural cost less critical for DeepCubeA. Our work shows that with the right feature representation, even a tiny MLP can achieve high prediction accuracy (83.3%), and that *input representation matters more than model scale* for this domain.

6.4 Deployment Recommendations

For most applications, K=3 offers the best trade-off (Table 12): 98% of the quality gain at 99% of the cost. The C backend makes even K=5 fast enough for real-time use ($<$50 ms).

7 Conclusion

We extended Kociemba's Two-Phase Algorithm with multi-solution search, neural move ordering, and formal analysis. Our main findings:

1. **Multi-solution search works.** K=5 reduces mean moves by 0.42 ($p < 0.001$, $d = 0.70$) and doubles $\leq$20-move solutions (28%$\rightarrow$56%), robust across four seeds.
2. **Representation is the bottleneck, not model size.** Binned coordinate features (161 dims) raise accuracy from 5.6% to 83.3% and yield 2.4$\times$ node reduction.

3. **Inference overhead is real but solvable.** Per-node neural inference negates the node savings in Python. LUT precomputation (32K entries, 576 KB) largely eliminates this overhead ($+7\%$ vs. $+19\%$).
4. **Naïve neural bounds are harmful.** Using neural predictions as IDA* lower bounds causes exponential blowup (Theorem 1). The correct integration is move ordering.

Future work. Incorporating Phase 2 coordinates into the LUT, using graph neural networks on the cubie adjacency structure, and parallelising Phase 1 search across multiple cores are all promising directions. We particularly recommend Phase 1 endpoint prediction—learning which G_1 states lead to short Phase 2 solutions—as a way to make multi-solution search smarter rather than just broader.

Reproducibility. All benchmarks are reproducible via: `python benchmark.py -scrambles 200 -compare-baseline -max-phase1 5 -seed 42 -v`. Results, models, and scripts are provided.

References

1. H. Kociemba, "Two-Phase Algorithm," [Online]. Available: `https://kociemba.org/cube.htm`.
2. T. Rokicki, H. Kociemba, M. Davidson, and J. Dethridge, "God's Number is 20," 2010. [Online]. Available: `http://www.cube20.org/`.
3. muodov/kociemba, "Python package: C and Python implementations of Kociemba's two-phase algorithm," [Online]. Available: `https://github.com/muodov/kociemba`.
4. F. Agostinelli, S. McAleer, A. Shmakov, and P. Baldi, "Solving the Rubik's Cube with Deep Reinforcement Learning and Search," *Nature Mach. Intell.*, vol. 1, pp. 356–363, 2019.
5. R. E. Korf, "Finding Optimal Solutions to Rubik's Cube Using Pattern Databases," in *Proc. AAAI Conf. Artif. Intell.*, 1997, pp. 700–705.
6. M. Thistlethwaite, "A 45-Move Algorithm for Rubik's Cube," unpublished manuscript, 1981.
7. R. Brunetto and O. Trunda, "Deep Heuristic-Learning in the Rubik's Cube Domain: An Experimental Evaluation," *Proc. ITAT*, pp. 57–64, 2017.
8. W. Shen, F. Trevizan, and S. Thiébaux, "Learning Domain-Independent Heuristics for Grounded and Lifted Planning," in *Proc. AAAI Conf. Artif. Intell.*, 2020, pp. 9883–9891.
9. P. Ferber, F. Geissmann, T. Helmert, *et al.*, "Neural Network Heuristics for Classical Planning: A Study of Hyperparameter Space," in *Proc. Eur. Conf. Artif. Intell. (ECAI)*, 2022, pp. 839–846.
10. A. Felner, R. E. Korf, and S. Hanan, "Additive Pattern Database Heuristics," *J. Artif. Intell. Res. (JAIR)*, vol. 22, pp. 279–318, 2004.
11. J. Fridrich, "CFOP Method," [Online]. Available: `https://www.speedsolving.com/wiki/index.php/CFOP_method`.
12. R. E. Korf, "Depth-First Iterative-Deepening: An Optimal Admissible Tree Search," *Artif. Intell.*, vol. 27, no. 1, pp. 97–109, 1985.
13. P. E. Hart, N. J. Nilsson, and B. Raphael, "A Formal Basis for the Heuristic Determination of Minimum Cost Paths," *IEEE Trans. Syst. Sci. Cybern.*, vol. 4, no. 2, pp. 100–107, 1968.
14. J. Pearl, *Heuristics: Intelligent Search Strategies for Computer Problem Solving*. Reading, MA: Addison-Wesley, 1984.
15. J. C. Culberson and J. Schaeffer, "Pattern Databases," *Comput. Intell.*, vol. 14, no. 3, pp. 318–334, 1998.

16. R. E. Korf and A. Felner, "Disjoint Pattern Database Heuristics," *Artif. Intell.*, vol. 134, no. 1–2, pp. 9–22, 2002.
17. D. Silver *et al.*, "Mastering the Game of Go with Deep Neural Networks and Tree Search," *Nature*, vol. 529, pp. 484–489, 2016.
18. D. Silver *et al.*, "Mastering the Game of Go Without Human Knowledge," *Nature*, vol. 550, pp. 354–359, 2017.
19. S. J. Arfaee, S. Zilles, and R. C. Holte, "Learning Heuristic Functions for Large State Spaces," *Artif. Intell.*, vol. 175, no. 16–17, pp. 2075–2098, 2011.
20. J. Schaeffer, "The History Heuristic and Alpha-Beta Search Enhancements in Practice," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 11, no. 11, pp. 1203–1212, 1989.
21. A. Reinefeld and T. A. Marsland, "Enhanced Iterative-Deepening Search," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 16, no. 7, pp. 701–710, 1994.
22. D. Joyner, *Adventures in Group Theory: Rubik's Cube, Merlin's Machine, and Other Mathematical Toys*, 2nd ed. Baltimore, MD: Johns Hopkins Univ. Press, 2008.
23. J. Cohen, *Statistical Power Analysis for the Behavioral Sciences*, 2nd ed. Hillsdale, NJ: Lawrence Erlbaum, 1988.
24. B. Efron and R. J. Tibshirani, *An Introduction to the Bootstrap*. New York: Chapman and Hall/CRC, 1993.
25. H. A. David and H. N. Nagaraja, *Order Statistics*, 3rd ed. Hoboken, NJ: Wiley, 2003.
26. G. Hinton, O. Vinyals, and J. Dean, "Distilling the Knowledge in a Neural Network," in *Proc. NeurIPS Deep Learn. Workshop*, 2015.
27. D. P. Kingma and J. Ba, "Adam: A Method for Stochastic Optimization," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2015.
28. E. D. Demaine, M. L. Demaine, S. Eisenstat, A. Lubiw, and A. Winslow, "Algorithms for Solving Rubik's Cubes," in *Proc. Eur. Symp. Algorithms (ESA)*, 2011, pp. 689–700.
29. A. Paszke *et al.*, "PyTorch: An Imperative Style, High-Performance Deep Learning Library," in *Advances in Neural Inf. Process. Syst. (NeurIPS)*, vol. 32, 2019.